MCT-244L: Signal Processing

Semester Project

False Coloration of BW Images

SUBMITTED BY:

FIDA HUSSAIN (2023-MC-42)

SYED AYAN ABBAS ZAIDI (2023-MC-61)

BASIL IFTIKHAR (2023-MC-66)

SYED IRTAZA ABBAS (2023-MC-70)

SUBMITTED TO:

MS. MALIHA BAKSHI

Department of Mechatronics and Control Engineering
University of Engineering and Technology, Lahore

CONTENTS

1. Introduction 2
2. Problem Statement 2
3. Objectives 3
4. Literature Review 3
5. Tools and Technologies 3
6. Methodology 3
6.1 Data Input & Preprocessing 3
6.2 False Coloration using Colormaps 3
6.3 Realistic Colorization using Deep Learning 4
6.4 GUI Implementation with PyQt5 5
7. Results and Discussion 6
7.1 Colormap Output Examples 6
7.2 Realistic Colorization 8
7.3 GUI Evaluation 9
8. Conclusion 10
9. Future Work 10
10. References 10
Appendix 11
Python code 11

1. INTRODUCTION

In many fields—such as medical imaging, satellite analysis, and digital art—color plays a critical role in perception and analysis. However, grayscale images lack this dimension. The aim of this project is to bridge that gap using signal processing principles by implementing both false and realistic colorization techniques. Grayscale images contain only intensity (luminance) information and lack chromatic (color) data. While sufficient for capturing structural and textural details, they often fall short in conveying semantic or contextual cues—particularly in applications such as medical imaging, satellite imagery, or historical photo restoration, where color can provide critical insights. Traditional approaches to adding color involve manual annotation and editing, which is not only labor-intensive but also prone to subjective bias and inconsistency.

This project aims to automate the colorization process by implementing a hybrid system that leverages both signal processing techniques (via predefined colormaps) and data-driven methods (via deep learning). The use of colormaps allows for rapid application of color gradients based on pixel intensity, which is useful for visualization and enhancement tasks. Meanwhile, deep learning models trained on large datasets can infer semantically appropriate colors for complex scenes, providing more natural and realistic colorization. By combining these approaches within an interactive GUI, the project offers a powerful tool for both exploratory visualization and practical restoration of black and white images.

2. PROBLEM STATEMENT

Grayscale images, while efficient in terms of storage and transmission, contain only luminance (brightness) information and completely lack chromatic (color) data. This limitation significantly reduces their effectiveness in scenarios where color plays a critical role in interpretation, classification, and analysis. For example, in medical imaging, color can help differentiate between tissues; in satellite imagery, it aids in distinguishing vegetation, water bodies, and urban areas; and in historical photo restoration, it brings life and context to otherwise flat and emotionless visuals.

Manual colorization of such images is a tedious, time-consuming, and highly subjective task that requires significant artistic skill and domain knowledge. Furthermore, it is not scalable when dealing with large datasets or real-time applications. Traditional rule-based techniques using colormaps can enhance visual appeal and highlight intensity differences but often lack semantic accuracy. On the other hand, fully automated deep learning models can produce photorealistic results but require complex setup and are sometimes not flexible for user interaction or adjustments.

This project addresses the dual challenge of (1) making grayscale images more informative and visually appealing through false colorization and (2) enabling realistic color generation that approximates natural scenes. By integrating both signal-based colormap techniques and deep learning-based colorization into a unified, user-friendly Python GUI application, the project aims to deliver a flexible, educational, and practical tool for researchers, students, and digital artists.

The solution not only automates the process but also provides customization and interactivity, making it suitable for a wide range of signal processing and computer vision applications.

3. OBJECTIVES

- Apply colormap-based false coloration to grayscale images
- Implement realistic colorization using a deep learning model (Caffe)
- Develop a PyQt5 GUI that allows user interaction with live preview
- Analyze the performance of different methods visually and qualitatively

4. LITERATURE REVIEW

False coloration has been used historically in thermal imaging, medical imaging (e.g., X-rays), and remote sensing. Colormaps like Jet, HSV, and Parula convert grayscale values to visible spectra using signal transformation techniques.

Recent advancements in deep learning have enabled the development of models like DeOldify and Caffe's colorization model, which learn context-aware color mappings from large datasets. These models infer the chrominance channels (ab) from luminance (L) in the Lab color space.

5. TOOLS AND TECHNOLOGIES

- **Language:** Python 3.9
- **Libraries:** OpenCV, NumPy, PyQt5
- **GUI Framework:** PyQt5
- **Deep Learning Framework:** OpenCV DNN module (Caffe model)
- **Model Files:**
  - `colorization_deploy_v2.prototxt`
  - `colorization_release_v2.caffemodel`
  - `pts_in_hull.npy`

6. METHODOLOGY

6.1 DATA INPUT & PREPROCESSING

The application begins by allowing the user to load any image from the file system. If the image is already in color, it is first converted to grayscale using OpenCV's cv2.cvtColor() method to simulate the false coloration process. This ensures consistency across all operations.

6.2 FALSE COLORATION USING COLORMAPS

False coloration involves mapping grayscale intensity values (ranging from 0 to 255) to color using predefined colormaps. OpenCV provides various colormaps such as:

cv2.COLORMAP_JET – creates a rainbow-like effect with red at high intensity and blue at low

cv2.COLORMAP_HOT – emphasizes bright features using warm tones (black to red to yellow to white)

cv2.COLORMAP_PARULA, COLORMAP_OCEAN, COLORMAP_BONE, etc.

IN THIS PROJECT:

The grayscale image is processed with the selected colormap using `cv2.applyColorMap()`.

The resulting colorized image is then blended with the original grayscale image (converted to BGR) using OpenCV's `addWeighted()` method.

A GUI slider allows the user to control the "blending intensity" (from 0 to 1), which determines how strongly the colormap affects the final image.

This method is fast and provides enhanced contrast and detail visibility, which is often useful in thermal, scientific, and artistic visualizations.

6.3 REALISTIC COLORIZATION USING DEEP LEARNING

To generate natural-looking color images from grayscale, we use a pre-trained deep learning model based on the **Caffe framework**. The approach involves the following steps:

CONVERT TO LAB COLOR SPACE

The grayscale image is first converted into the Lab color space using cv2.cvtColor(). Lab separates luminance (L) from chrominance (a, b), which is ideal for color prediction.

EXTRACT L CHANNEL

The L (lightness) channel is resized to 224×224 pixels and normalized to a range of [0,1]. This resized L channel is fed into the deep neural network.

LOAD PRE-TRAINED CAFFE MODEL

The model architecture `(colorization_deploy_v2.prototxt)` and weights `(colorization_release_v2.caffemodel)` are loaded using OpenCV's `cv2.dnn.readNetFromCaffe()` function. The `pts_in_hull.npy` file is also used to initialize cluster centers for the ab color channels.

PREDICT AB CHANNELS

The model infers the 'a' and 'b' channels (representing color) from the input L channel.

RECONSTRUCT COLOR IMAGE

The predicted ab channels are upsampled to match the original image size and merged with the L channel. The Lab image is then converted back to BGR (color) using OpenCV.

DISPLAY AND SAVE OUTPUT

The result is displayed in the GUI and can be saved for later use.

This method generates context-aware, semantically plausible colors and is effective for photos with people, nature, or urban scenes. However, it may struggle with abstract or heavily modified grayscale images.

6.4 GUI IMPLEMENTATION WITH PYQT5

The graphical user interface (GUI) is built using PyQt5 and includes the following components:

Image Display Panels: Two panels show the original grayscale image and the processed (false-colored or realistically colorized) image side-by-side.

File Operations:

Load Image – allows user to browse and select an image.

Save Output – saves the processed image.

Colormap Selection:
A dropdown menu enables selection from various OpenCV colormaps.

Intensity Slider:
A horizontal slider lets users adjust the blending intensity between the colormap output and the original image.

Realistic Colorization Button:
A button labeled “Realistic Colorization” triggers the deep learning model to colorize the image semantically.

Progress Indicator:
A progress bar at the bottom provides visual feedback while the model loads or processes the image.

This GUI makes the application user-friendly, allowing users with no coding experience to explore signal processing techniques interactively.

7. RESULTS AND DISCUSSION

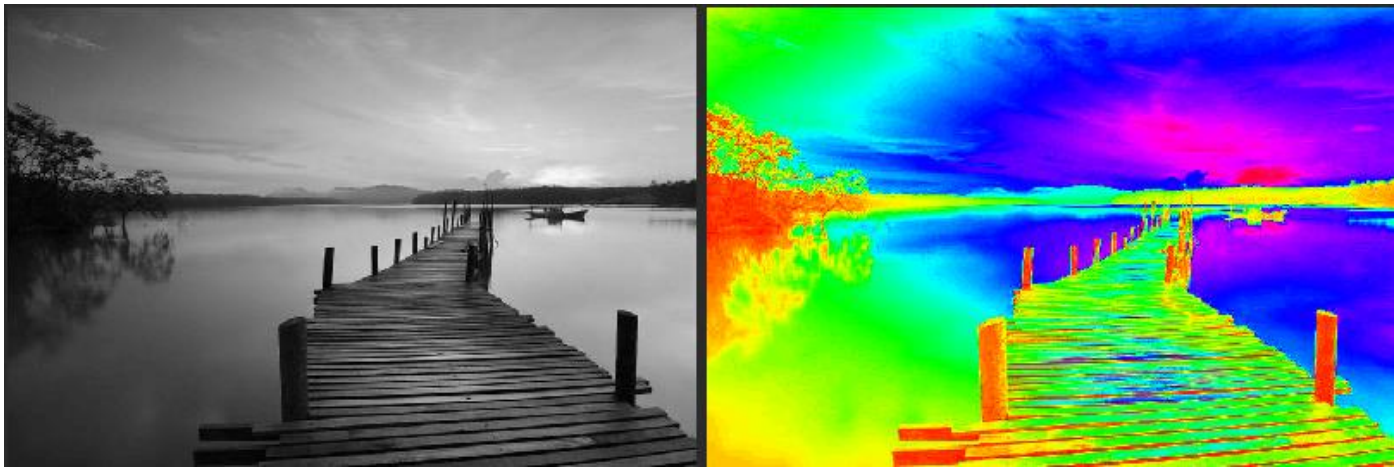
7.1 COLORMAP OUTPUT EXAMPLES

Each colormap produces visually distinct results:

- **Jet:** High contrast and sharp transitions

- **Parula:** Smooth gradients

- **HSV:** Ideal for phase representation



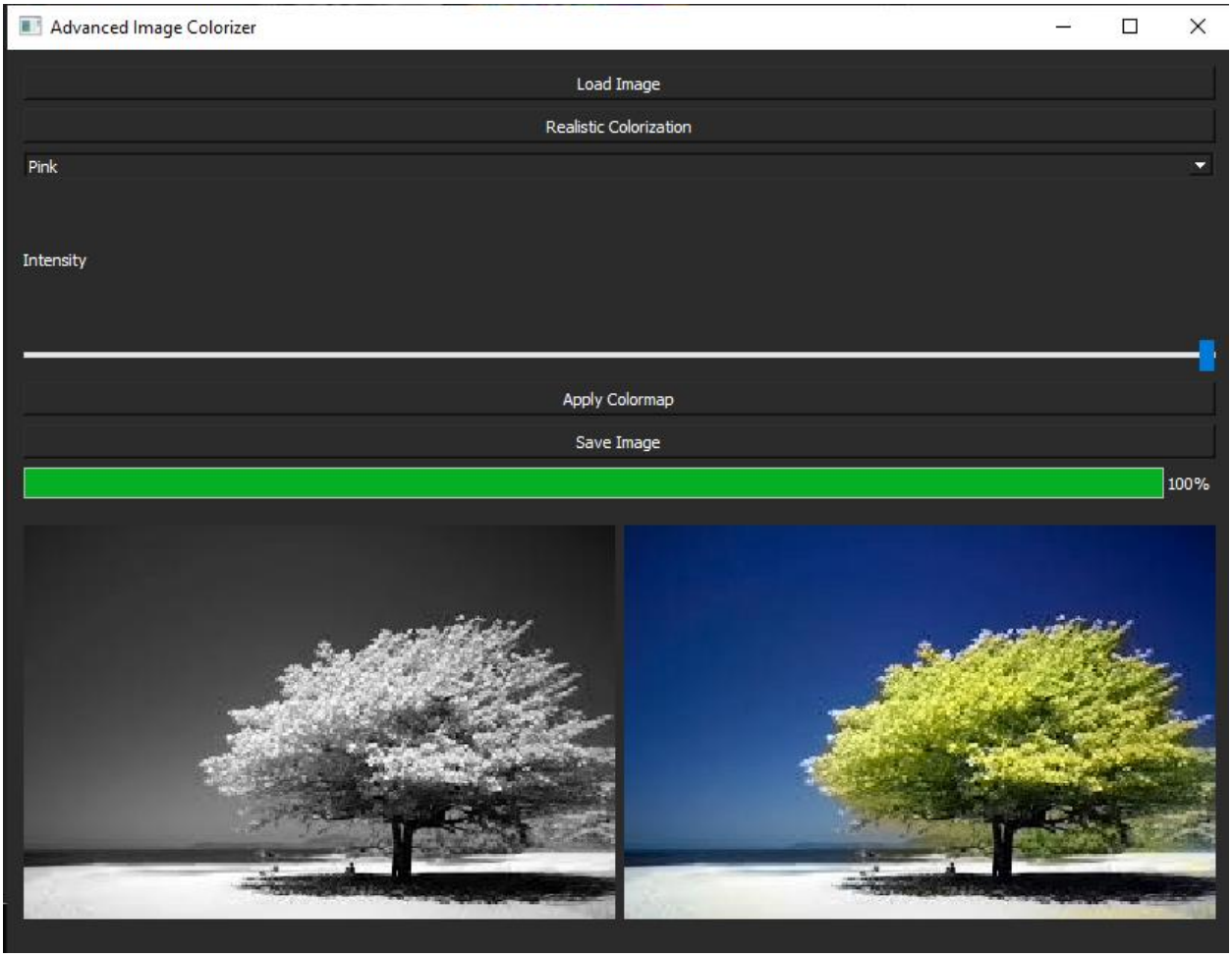
- **Bone & Pink:** Softer tones for medical imaging aesthetics

7.2 REALISTIC COLORIZATION

The deep learning model performed well on natural scenes but showed limitations on synthetic or abstract grayscale images. In some cases, the colorization was semantically accurate (e.g., skies turned blue, grass green).

7.3 GUI EVALUATION

The GUI was tested for usability and functionality. Real-time previews, save/load features, and progress indicators contributed to a smooth user experience.



8. CONCLUSION

This project successfully developed a comprehensive image colorization system that combines traditional signal-processing techniques with advanced deep learning-based methods. By integrating multiple colormaps (such as Jet, Parula, and Hot) alongside a realistic colorization pipeline powered by a Caffe-based convolutional neural network, the application bridges the gap between classical and modern approaches to image enhancement.

The user-friendly PyQt5 GUI not only enables intuitive interaction but also provides an experimental platform for students and practitioners to explore the practical applications of signal processing in image manipulation. The inclusion of adjustable intensity for colormaps and side-by-side image comparisons further enhances its educational and practical value.

This dual-approach system demonstrates how signal-based transformations and data-driven models can be harmoniously used to restore, analyze, and enhance black-and-white images — a concept central to many signal processing and computer vision problems today.

9. FUTURE WORK

- Integrate video colorization in real time
- Allow user-guided color hints (scribbles or reference images)
- Implement histogram equalization and contrast enhancement
- Web deployment using Flask or Django

10. REFERENCES

[1] YouTube, "Image Colorization with Deep Learning," *YouTube Video*, 2023. [Online]. Available: https://youtu.be/gAmskBNz_Vc?si=D_t-mkBfdWQnozwG

[2] R. Zhang, "colorization_deploy_v2.prototxt," *GitHub Repository*. [Online]. Available: https://github.com/richzhang/colorization/blob/master/models/colorization_deploy_v2.prototxt

[3] R. Zhang, "pts_in_hull.npy," *GitHub Repository*. [Online]. Available: https://github.com/richzhang/colorization/blob/master/resources/pts_in_hull.npy

[4] R. Zhang, "colorization_release_v2.caffemodel," *Dropbox*. [Online]. Available: https://www.dropbox.com/s/dx0qvhhp5hb...

[5] OpenCV, "Open Source Computer Vision Library," [Online]. Available: https://opencv.org/

[6] PyQt5, "PyQt5 Documentation," [Online]. Available: https://www.riverbankcomputing.com/software/pyqt/intro

[7] OpenAI, "ChatGPT," [Online]. Available: https://chat.openai.com/, accessed May 29, 2025.

[8] DeepSeek, “DeepSeek Chat - AI Assistant,” [Online]. Available: https://chat.deepseek.com/, accessed May 29, 2025.

APPENDIX

PYTHON CODE

```
import sys
import os
import numpy as np
import cv2
from PyQt5.QtWidgets import (QApplication, QWidget, QLabel, QPushButton, QFileDialog,
QVBoxLayout, QHBoxLayout,
                             QComboBox, QSlider, QProgressBar, QSpacerItem, QSizePolicy,
QMessageBox)
from PyQt5.QtCore import Qt
from PyQt5.QtGui import QPixmap, QImage

class ImageColorizerApp(QWidget):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Advanced Image Colorizer")
        self.setStyleSheet("background-color: #2b2b2b; color: white;")

        self.original_label = QLabel("Original Image")
        self.original_label.setAlignment(Qt.AlignCenter)
        self.colored_label = QLabel("Colored Image")
        self.colored_label.setAlignment(Qt.AlignCenter)

        self.colormap_box = QComboBox()
        self.colormap_box.addItems([
            "Jet", "Parula", "Hot", "Cool", "Spring", "Summer",
            "Autumn", "Winter", "Bone", "HSV", "Pink", "Gray"
        ])

        self.intensity_slider = QSlider(Qt.Horizontal)
        self.intensity_slider.setMinimum(0)
        self.intensity_slider.setMaximum(100)
        self.intensity_slider.setValue(100)
        self.intensity_slider.setTickInterval(10)

        self.load_button = QPushButton("Load Image")
        self.realistic_button = QPushButton("Realistic Colorization")
        self.apply_colormap_button = QPushButton("Apply Colormap")
        self.save_button = QPushButton("Save Image")
        self.progress_bar = QProgressBar()
```

```
        layout = QVBoxLayout()
        layout.addWidget(self.load_button)
        layout.addWidget(self.realistic_button)
        layout.addWidget(self.colormap_box)
        layout.addWidget(QLabel("Intensity"))
        layout.addWidget(self.intensity_slider)
        layout.addWidget(self.apply_colormap_button)
        layout.addWidget(self.save_button)
        layout.addWidget(self.progress_bar)

        image_layout = QHBoxLayout()
        image_layout.addWidget(self.original_label)
        image_layout.addWidget(self.colored_label)

        main_layout = QVBoxLayout()
        main_layout.addLayout(layout)
        main_layout.addLayout(image_layout)
        self.setLayout(main_layout)

        self.load_button.clicked.connect(self.load_image)
        self.realistic_button.clicked.connect(self.colorize_realistically)
        self.apply_colormap_button.clicked.connect(self.apply_colormap)
        self.save_button.clicked.connect(self.save_image)

        self.image = None
        self.colored_image = None

    def load_image(self):
        file_name, _ = QFileDialog.getOpenFileName(self, "Open Image", "", "Images (*.png
*.xpm *.jpg *.jpeg *.bmp)")
        if file_name:
            self.image = cv2.imread(file_name, cv2.IMREAD_COLOR)
            self.display_image(self.image, self.original_label)
            self.progress_bar.setValue(0)
            self.colored_image = None

    def display_image(self, image, label):
        height, width, channel = image.shape
        bytes_per_line = 3 * width
        q_img = QImage(image.data, width, height, bytes_per_line,
QImage.Format_RGB888).rgbSwapped()
        pixmap = QPixmap.fromImage(q_img).scaled(400, 400, Qt.KeepAspectRatio)
        label.setPixmap(pixmap)

    def colorize_realistically(self):
        if self.image is None:
            return
```

```
        self.progress_bar.setValue(10)
        image_path = os.path.join(os.getcwd(), "temp_image.jpg")
        cv2.imwrite(image_path, self.image)
        self.colored_image = self.colorize_image_realistically(image_path)
        self.display_image(self.colored_image, self.colored_label)
        self.progress_bar.setValue(100)

    def colorize_image_realistically(self, image_path):
        DIR = r"C:\\Users\\Administrator\\OneDrive\\Desktop\\Signal Processing\\Project"
        proto = os.path.join(DIR, "colorization_deploy_v2.prototxt")
        model = os.path.join(DIR, "colorization_release_v2.caffemodel")
        points = os.path.join(DIR, "pts_in_hull.npy")

        net = cv2.dnn.readNetFromCaffe(proto, model)
        pts = np.load(points)
        pts = pts.transpose().reshape(2, 313, 1, 1)
        net.getLayer(net.getLayerId("class8_ab")).blobs = [pts.astype("float32")]
        net.getLayer(net.getLayerId("conv8_313_rh")).blobs = [np.full([1, 313], 2.606,
dtype="float32")]

        image = cv2.imread(image_path)
        scaled = image.astype("float32") / 255.0
        lab = cv2.cvtColor(scaled, cv2.COLOR_BGR2LAB)
        resized = cv2.resize(lab, (224, 224))
        L = cv2.split(resized)[0]
        L -= 50
        net.setInput(cv2.dnn.blobFromImage(L))
        ab = net.forward()[0, :, :, :].transpose((1, 2, 0))
        ab = cv2.resize(ab, (image.shape[1], image.shape[0]))
        L = cv2.split(lab)[0]
        colorized = np.concatenate((L[:, :, np.newaxis], ab), axis=2)
        colorized = cv2.cvtColor(colorized, cv2.COLOR_LAB2BGR)
        colorized = np.clip(colorized, 0, 1)
        return (255 * colorized).astype("uint8")

    def apply_colormap(self):
        if self.image is None:
            return

        self.progress_bar.setValue(20)
        gray = cv2.cvtColor(self.image, cv2.COLOR_BGR2GRAY)
        selected_map = self.colormap_box.currentText().lower()
        intensity = self.intensity_slider.value() / 100.0

        colormap_dict = {
            "jet": cv2.COLORMAP_JET,
            "parula": cv2.COLORMAP_PARULA,
```

```
            "hot": cv2.COLORMAP_HOT,
            "cool": cv2.COLORMAP_COOL,
            "spring": cv2.COLORMAP_SPRING,
            "summer": cv2.COLORMAP_SUMMER,
            "autumn": cv2.COLORMAP_AUTUMN,
            "winter": cv2.COLORMAP_WINTER,
            "bone": cv2.COLORMAP_BONE,
            "hsv": cv2.COLORMAP_HSV,
            "pink": cv2.COLORMAP_PINK,
            "gray": -1
        }

        cmap = colormap_dict.get(selected_map, cv2.COLORMAP_JET)
        if cmap == -1:
            colored = cv2.cvtColor(gray, cv2.COLOR_GRAY2BGR)
        else:
            colored = cv2.applyColorMap(gray, cmap)

        blended = cv2.addWeighted(colored, intensity, cv2.cvtColor(gray,
cv2.COLOR_GRAY2BGR), 1 - intensity, 0)
        self.colored_image = blended
        self.display_image(blended, self.colored_label)
        self.progress_bar.setValue(100)

    def save_image(self):
        if self.colored_image is None:
            QMessageBox.warning(self, "Warning", "No colored image to save!")
            return

        file_name, _ = QFileDialog.getSaveFileName(
            self, "Save Image", "", "Images (*.png *.jpg *.jpeg *.bmp)")

        if file_name:
            # Convert BGR to RGB for proper color saving
            save_image = cv2.cvtColor(self.colored_image, cv2.COLOR_BGR2RGB)
            success = cv2.imwrite(file_name, save_image)

            if not success:
                QMessageBox.critical(self, "Error", "Failed to save image!")
            else:
                QMessageBox.information(self, "Success", "Image saved successfully!")

app = QApplication(sys.argv)
window = ImageColorizerApp()
window.show()
sys.exit(app.exec_())
```